

CloudMailExchangeService

Exchange Online (Microsoft 365) mailbox client

EWS + ADAL4J (ROPC)



Microsoft Graph + Azure Identity (app-only)

notification-service · com.dbs.edoc.notification.services.notification.ews · Spring Boot / Java 17
EWS for Exchange Online begins being blocked 1 Oct 2026, fully shut down 1 Apr 2027

1. Role of this Class

CloudMailExchangeService connects to **Exchange Online** (cloud, Microsoft 365) mailboxes to send outbound notification emails through the corporate proxy. It authenticates against Azure AD and hands a ready-to-use client object back to the shared `MailExchangeService.saveMailToOutbox(...)` logic. This is exactly the class the EWS retirement targets: Exchange Online is where EWS is being switched off.

Security bonus, not just a compliance checkbox

The current implementation uses **ADAL4J's Resource-Owner-Password-Credentials (ROPC) grant** — it passes the mailbox's own **username and password** straight to Azure AD to get a token. ROPC is itself a legacy/deprecated OAuth2 flow that Microsoft discourages (it's incompatible with MFA and Conditional Access). The Graph migration below switches to the **client-credentials (app-only) flow**: the application authenticates as itself with a registered `client_id` + `client_secret`, and is granted an application permission (`Mail.Send`) to send as any mailbox in the tenant. Mailbox passwords are no longer read, stored, decrypted, or transmitted by this service at all.

2. Before Migration — EWS + ADAL4J (ROPC)

CloudMailExchangeService.java – CURRENT (EWS, deprecated path)

```
package com.dbs.edoc.notification.services.notification.ews;

import com.dbs.edoc.config.DynamicIntProperty;
import com.dbs.edoc.config.DynamicLongProperty;
import com.dbs.edoc.config.DynamicStringProperty;
import com.dbs.edoc.notification.error.ExchangeMessageException;
import com.dbs.edoc.notification.util.ProxyRegistry;
import com.google.common.cache.Cache;
import com.google.common.cache.CacheBuilder;
import com.microsoft.aad.adal4j.AuthenticationContext;
import com.microsoft.aad.adal4j.AuthenticationResult;
import microsoft.exchange.webservices.data.core.ExchangeService;
import microsoft.exchange.webservices.data.core.WebProxy;
import microsoft.exchange.webservices.data.core.enumeration.misc.ExchangeVersion;
import microsoft.exchange.webservices.data.credential.WebCredentials;
import microsoft.exchange.webservices.data.credential.WebProxyCredentials;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.stereotype.Service;

import java.net.URI;
import java.util.*;
import java.util.concurrent.*;

@Service
public class CloudMailExchangeService extends MailExchangeService {

    private static final Logger LOGGER = LoggerFactory.getLogger(CloudMailExchangeService.class);

    private ExchangeService exchangeService = new ExchangeService(ExchangeVersion.Exchange2010_SP2);
    private ExecutorService executorService = Executors.newSingleThreadExecutor();

    private static final DynamicStringProperty CLIENT_ID = new DynamicStringProperty("cloud.exchange.client.id", "");
    private static final DynamicStringProperty TENANT_ID = new DynamicStringProperty("cloud.exchange.tenant.id", "");
    private static final DynamicStringProperty AUTHORITY = new DynamicStringProperty("cloud.exchange.authority.url", "https://login.microsoftonline.com/");
    private static final DynamicStringProperty RESOURCE_URL = new DynamicStringProperty("cloud.exchange.resource.url", "https://outlook.office365.com");
    private static final DynamicStringProperty EXCHANGE_SERVICE_URL = new DynamicStringProperty("cloud.exchange.service.url",
"https://outlook.office365.com/EWS/Exchange.asmx");
    private static final DynamicLongProperty TOKEN_EXPIRY_MINS = new DynamicLongProperty("cloud.exchange.token.expiry.mins", 200);
    private static final DynamicIntProperty EXCHANGE_SERVICE_TIMEOUT = new DynamicIntProperty("cloud.exchange.service.timeout", 30000);
    private static final DynamicLongProperty OUTLOOK_TIMEOUT = new DynamicLongProperty("cloud.exchange.outlook.timeout.seconds", 30);

    private final Cache<String, String> tokens;

    public CloudMailExchangeService() {

        tokens = CacheBuilder.newBuilder()
            .expireAfterWrite(TOKEN_EXPIRY_MINS.getValue(), TimeUnit.MINUTES)
            .build();

        try {
            ProxyRegistry.authenticateProxy();
            WebProxy webProxy = new WebProxy(ProxyRegistry.getHost(), Integer.parseInt(ProxyRegistry.getPort()), new
WebProxyCredentials(ProxyRegistry.getUsername(), ProxyRegistry.getPassword(), null));
            exchangeService.setWebProxy(webProxy);
            exchangeService.setTimeout(EXCHANGE_SERVICE_TIMEOUT.getValue());
            exchangeService.setUrl(new URI(EXCHANGE_SERVICE_URL.getValue()));

            LOGGER.info("Initialized Cloud Mail Exchange Service");
        } catch (Exception ex) {
            LOGGER.error("Error occurred while initializing Cloud Mail Exchange Service", ex);
        }
    }

    @Override
    public void initializeExchangeService(String email, String password, String domain) throws ExchangeMessageException {
        ProxyRegistry.registerProxySettings();
        LOGGER.info("Initializing Exchange Web Service with User name [{}]", email);
        String firstName = email.split("@")[0].trim();
        LOGGER.info("First name extracted [{}] as Web credentials", firstName);
        exchangeService.setCredentials(new WebCredentials(firstName, password));

        final String authKey = tokens.getIfPresent(email);
        if (authKey == null) {
            String accessToken = acquireToken(email, password);
            if (accessToken != null) {
                tokens.put(email, accessToken);
                exchangeService.getHttpHeaders().put("Authorization", "Bearer " + accessToken);
                LOGGER.info("New Access Token updated in Headers successfully!");
            } else {
                LOGGER.error("Could not acquire access Token for Email [{}]", email);
            }
        } else {
            exchangeService.getHttpHeaders().put("Authorization", "Bearer " + authKey);
            LOGGER.info("Existing Access Token updated in Headers with successfully!");
        }
    }

    private String acquireToken(String email, String password) throws ExchangeMessageException {
        try {
            LOGGER.info("Authority URL [{}]", AUTHORITY.getValue() + TENANT_ID.getValue());
            AuthenticationContext context = new AuthenticationContext(AUTHORITY.getValue() + TENANT_ID.getValue(), false, executorService);
            context.setLogPii(true);
            Future<AuthenticationResult> authenticationResultFuture = context.acquireToken(RESOURCE_URL.getValue(), CLIENT_ID.getValue(), email, password,
null);

            final AuthenticationResult authenticationResult = authenticationResultFuture.get(OUTLOOK_TIMEOUT.getValue(), TimeUnit.SECONDS);
            LOGGER.info("AuthenticationResult received : [{}]", authenticationResult);
        }
    }
}
```

```
        if (authenticationResult == null) {
            LOGGER.error(" Unable to fetch AuthenticationResult !! ");
        } else {
            final Date expiresOnDate = authenticationResult.getExpiresOnDate();
            String accessToken = authenticationResult.getAccessToken();
            LOGGER.info("Access Token found [{}] for Email [{}]", accessToken, email);
            LOGGER.info("Expiry Date {}", expiresOnDate);
            if (!accessToken.isEmpty()) {
                return accessToken;
            } else {
                LOGGER.error(" Unable to fetch accessToken! ");
            }
        }
    }
} catch (Exception e) {
    LOGGER.error("Error occurred acquiring Token", e);
    throw new ExchangeMessageException("Error occurred acquiring Token", e);
}

return null;
}

@Override
public ExchangeService getExchangeService() { return this.exchangeService; }
```

3. After Migration — Microsoft Graph + Azure Identity

CloudMailExchangeService.java – MIGRATED (Microsoft Graph, app-only auth)

```
package com.dbs.edoc.notification.services.notification.ews;

import com.azure.identity.ClientSecretCredential;
import com.azure.identity.ClientSecretCredentialBuilder;
import com.dbs.edoc.config.DynamicStringProperty;
import com.dbs.edoc.notification.error.ExchangeMessageException;
import com.dbs.edoc.notification.util.ProxyRegistry;
import com.microsoft.graph.serviceclient.GraphServiceClient;
import okhttp3.Credentials;
import okhttp3.OkHttpClient;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.stereotype.Service;

import java.net.InetSocketAddress;
import java.net.Proxy;

@Service
public class CloudMailExchangeService extends MailExchangeService {

    private static final Logger LOGGER = LoggerFactory.getLogger(CloudMailExchangeService.class);

    // GraphServiceClient is thread-safe and cheap to reuse, so one instance
    // for the whole service (built once in the constructor) replaces the old
    // single ExchangeService + hand-rolled token Cache.
    private GraphServiceClient graphServiceClient;

    private static final DynamicStringProperty CLIENT_ID = new DynamicStringProperty("cloud.exchange.client.id", "");
    private static final DynamicStringProperty CLIENT_SECRET = new DynamicStringProperty("cloud.exchange.client.secret", "");
    private static final DynamicStringProperty TENANT_ID = new DynamicStringProperty("cloud.exchange.tenant.id", "");

    // Graph's default scope for application (client-credentials) permissions.
    // Requires the "Mail.Send" Application permission, admin-consented once
    // in Azure AD -- see the "Azure AD App Registration" section below.
    private static final String GRAPH_DEFAULT_SCOPE = "https://graph.microsoft.com/.default";

    public CloudMailExchangeService() {
        try {
            ProxyRegistry.authenticateProxy();

            OkHttpClient.Builder httpClientBuilder = new OkHttpClient.Builder();
            if (ProxyRegistry.getHost() != null && !ProxyRegistry.getHost().isEmpty()) {
                httpClientBuilder.proxy(new Proxy(Proxy.Type.HTTP,
                    new InetSocketAddress(ProxyRegistry.getHost(), Integer.parseInt(ProxyRegistry.getPort()))));
                httpClientBuilder.proxyAuthenticator((route, response) -> response.request().newBuilder()
                    .header("Proxy-Authorization",
                        Credentials.basic(ProxyRegistry.getUsername(), ProxyRegistry.getPassword()))
                    .build());
            }
            OkHttpClient httpClient = httpClientBuilder.build();

            // Client-credentials (app-only) OAuth2 flow: the app authenticates
            // as itself using CLIENT_ID + CLIENT_SECRET + TENANT_ID -- no
            // mailbox username/password is used anywhere in this flow.
            ClientSecretCredential credential = new ClientSecretCredentialBuilder()
                .clientId(CLIENT_ID.getValue())
                .clientSecret(CLIENT_SECRET.getValue())
                .tenantId(TENANT_ID.getValue())
                .build();

            this.graphServiceClient = new GraphServiceClient(credential, httpClient, GRAPH_DEFAULT_SCOPE);

            LOGGER.info("Initialized Cloud Mail Exchange Service (Microsoft Graph, app-only auth)");
        } catch (Exception ex) {
            LOGGER.error("Error occurred while initializing Cloud Mail Exchange Service", ex);
        }
    }

    @Override
    void initializeExchangeService(String email, String password, String domain) throws ExchangeMessageException {
        // Intentionally a no-op. With application permissions, the single
        // graphServiceClient built above can already send mail "as" any
        // mailbox in the tenant that the Mail.Send permission covers -- there
        // is no per-user login, no acquireToken() call, and no mailbox
        // password to manage at all. The (email, password, domain) signature
        // is kept only because MailExchangeService.saveMailToOutbox(...)
        // still calls it -- see the base-class migration notes.
        LOGGER.info("Application-only Graph auth in use; no per-user token needed for [{}]", email);
    }

    @Override
    GraphServiceClient getGraphServiceClient() {
        return graphServiceClient;
    }
}
```

4. What Actually Changed, and Why

Concern	Before (EWS + ADAL4J)	After (Graph + Azure Identity)
Client library	ews-java-api (ExchangeService, ExchangeVersion.Exchange2010_SP2)	microsoft-graph SDK (GraphServiceClient)
OAuth2 grant type	Resource Owner Password Credentials (ROPC) via ADAL4J — app collects and forwards the mailbox password	Client Credentials (app-only) via ClientSecretCredential — app authenticates as itself, no mailbox password involved
Auth library	adal4j (ADAL, deprecated by Microsoft in favour of MSAL since 2020)	azure-identity (current, actively maintained, wraps MSAL4J internally)
Token acquisition	Manual acquireToken(email, password) method calling AuthenticationContext.acquireToken(...)	Removed entirely — ClientSecretCredential fetches tokens on demand internally
Token caching	Hand-rolled Guava Cache<String, String> keyed by email, manual expiry management	Removed — azure-identity caches and refreshes tokens internally; no custom cache code to maintain
Where the Bearer token is attached	Manually put into exchangeService.getHttpHeaders().put("Authorization", "Bearer " + token)	Handled transparently by the Graph SDK's request adapter using the supplied credential
Proxy configuration	WebProxy + WebProxyCredentials set directly on ExchangeService	Configured once on the underlying OkHttpClient (proxy + proxy authenticator), then injected into GraphServiceClient
initializeExchangeService(email, password, domain)	Does real work every call: sets per-user WebCredentials, checks cache, acquires/attaches a token	Effectively a no-op / logging call — kept only so the shared base-class contract is satisfied; app-only auth means one client instance already serves every mailbox
Endpoint / URL config	cloud.exchange.service.url → https://outlook.office365.com/EWS/Exchange.asmx	Not needed — the Graph SDK talks to https://graph.microsoft.com by default

5. Azure AD App Registration (one-time setup)

- **Register an application** in Azure AD (Entra ID) — note the generated **Application (client) ID** and **Directory (tenant) ID**.
- **Create a client secret** (or, preferably for production, a certificate credential) under "Certificates & secrets".
- **Grant an Application permission** (not Delegated) of Mail.Send under Microsoft Graph API permissions.
- **Admin-consent** the permission — application permissions always require tenant-admin consent.
- Optional but recommended: scope down which mailboxes the app can send as, using an **application access policy** (New-ApplicationAccessPolicy in Exchange Online PowerShell) restricted to a mail-enabled security group containing only the notification mailboxes this service actually uses.

6. Configuration Changes (application.properties)

Remove	Add
<pre>cloud.exchange.authority.url=https://login.microsoftonline.com/ cloud.exchange.resource.url=https://outlook.office365.com cloud.exchange.service.url=https://outlook.office365.com/EWS/Exchange.asmx cloud.exchange.service.timeout=30000 cloud.exchange.token.expiry.mins=200 cloud.exchange.outlook.timeout.seconds=30</pre>	<pre>cloud.exchange.client.id=<app-registration-client-id> cloud.exchange.client.secret=<app-registration-secret> # externalize via vault/KMS cloud.exchange.tenant.id=<azure-ad-tenant-id></pre>

Secret handling

cloud.exchange.client.secret must never sit in plaintext in an application.properties file committed to source control. Route it through the same vault/KMS mechanism already used for other production credentials, exactly as the current mailbox password is presumably protected today.

7. Maven Dependency Changes (pom.xml)

Remove	Add
<pre><dependency> <groupId>com.microsoft.ews-java-api</groupId> <artifactId>ews-java-api</artifactId> </dependency> <dependency> <groupId>com.microsoft.aad</groupId> <artifactId>adal4j</artifactId> </dependency> <dependency> <groupId>com.google.guava</groupId> <artifactId>guava</artifactId> </dependency> </dependency> <!-- only if used solely for the token Cache --></pre>	<pre><dependency> <groupId>com.microsoft.graph</groupId> <artifactId>microsoft-graph</artifactId> <version>6.20.0</version> </dependency> <dependency> <groupId>com.azure</groupId> <artifactId>azure-identity</artifactId> <version>1.13.3</version> </dependency></pre>